

GitHub Webhook

To Begin with the Lab

Summary of the Lab

This lab demonstrates how to configure a **GitHub webhook to automatically trigger Jenkins builds**. First, create a public GitHub repository named **github-webhook**. In Jenkins, create a new job, configure **Git** under Source Code Management using the repository URL, and enable “**GitHub hook trigger for GITScm polling**” in Build Triggers. Add build steps using **Execute Shell** or Maven goals such as **compile, test, and package**, then save and run the job. Next, add a webhook in the GitHub repository settings with the Jenkins payload URL (<http://<JENKINS-IP>:8080/github-webhook/>). When code changes are committed to GitHub, the webhook sends an event to Jenkins, which automatically triggers a new build.

- Go to **GitHub**.
- Click **New Repository**.
- Enter repository details:
 - Repository Name: github-webhook
 - Visibility: Public
- Go to Jenkins Dashboard.
- Click New Item.

New Item

Enter an item name

github-webhook

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

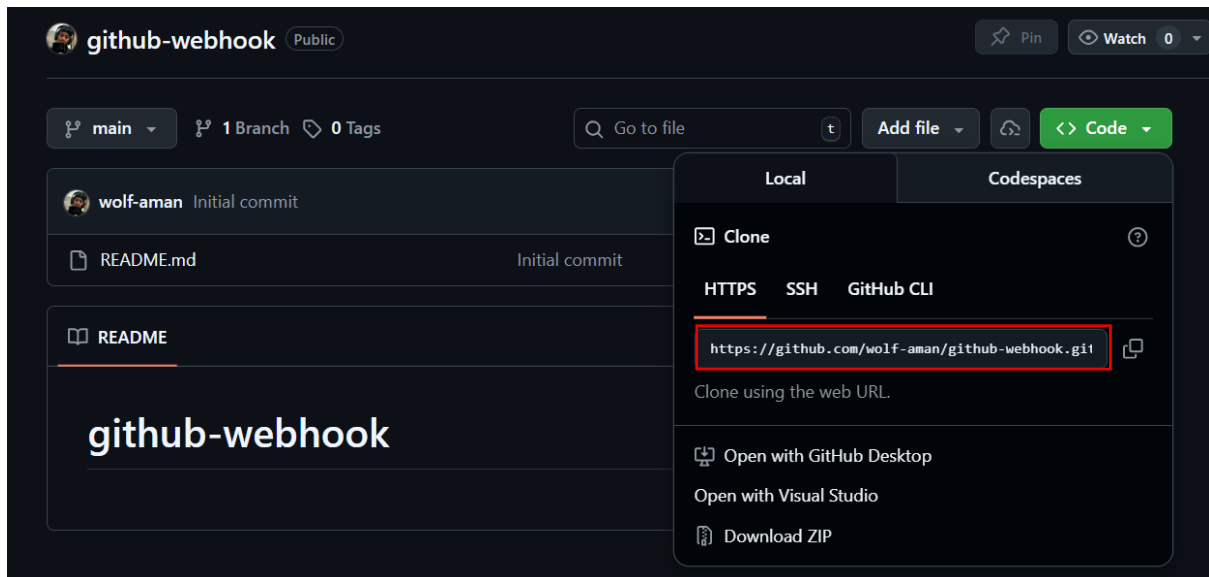
- Navigate Source Code Management
- Select the git option from there

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

- ☐ None
- ☒ Git ?

- After selecting git.
- Copy the URL from the github



- Enter the Repository URL
- Note: If your repository is private, you have to select the credentials from the credential option.

Repositories ?

Repository URL ?

`https://github.com/wolf-aman/github-webhook.git`

Credentials ?

- none -

+ Add

Advanced ▾

- Check you github repository is master or main.

Branch Specifier (blank for 'any') ?

`*/main`

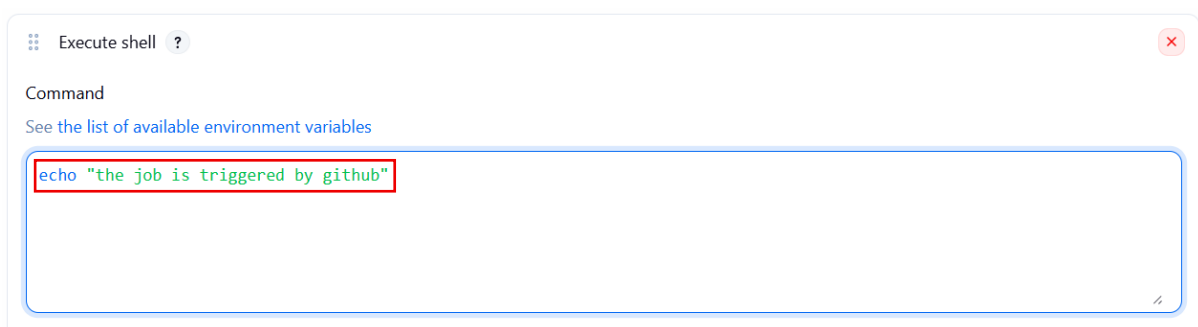
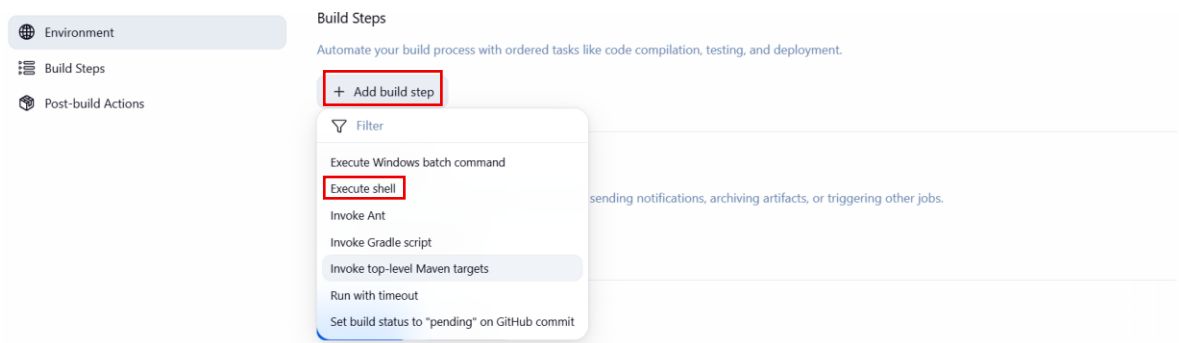
- Scroll to **Build Triggers**
- Enable the option: GitHub hook trigger for GITScm polling
- This allows Jenkins to **trigger builds when GitHub sends a webhook event**.

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

- Navigate to the Build Step section
- Click on the Add Build Step
- Select the 'Execute shell'

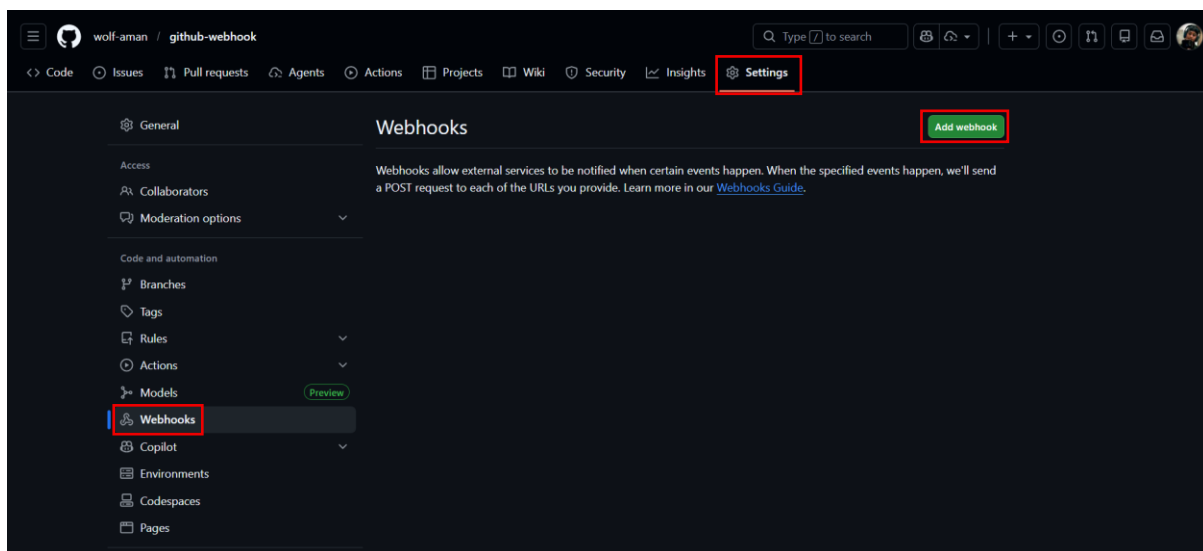


- Similarly make 3 Goal – compile, test and package with the Maven.
- Save the Job
- After saving it will create a job successfully
- Now click on the Build now
- Check the console output

```
Console Output

Started by user User1
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/github-webhook
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/wolf-aman/github-webhook.git
> git init /var/lib/jenkins/workspace/github-webhook # timeout=10
Fetching upstream changes from https://github.com/wolf-aman/github-webhook.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/wolf-aman/github-webhook.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git config remote.origin.url https://github.com/wolf-aman/github-webhook.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 4a40d37abbd3ea93f151ec6ee9facf9066950cad (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 4a40d37abbd3ea93f151ec6ee9facf9066950cad # timeout=10
Commit message: "Initial commit"
First time build. Skipping changelog.
[github-webhook] $ /bin/sh -xe /tmp/jenkins14305284270811756841.sh
+ echo 'the job is triggered by github'
the job is triggered by github
Finished: SUCCESS
```

- Configure GitHub Webhook
- Go to your **GitHub repository**.
- Click Setting > Select Webhooks > Click Add Webhook.



- Fill the following details:
- Payload URL- `http://<JENKINS-IP>:8080/github-webhook/`
- Example: `http://54.210.10.45:8080/github-webhook/`
- Content Type: `application/json`
- Click: Add Webhook

Code and automation

- Branches
- Tags
- Rules
- Actions
- Models
- Webhooks**
- Copilot
- Environments
- Codespaces
- Pages

Security

- Advanced Security
- Deploy keys
- Secrets and variables

Integrations

- GitHub Apps
- Email notifications
- Autolink references

Payload URL *
http://3.216.30.120:8080/github-webhook/

Content type *
application/json

Secret

SSL verification
By default, we verify SSL certificates when delivering payloads.
☒ Enable SSL verification ☐ Disable (not recommended)

Which events would you like to trigger this webhook?
☒ Just the push event.
☐ Send me everything.
☐ Let me select individual events.

☒ **Active**
We will deliver event details when this hook is triggered.

Add webhook

- Verify Webhook
- Click the created webhook.
- Go to: Recent Deliveries
- If configured correctly you will see: Status: 200 OK
- This means GitHub successfully connected to Jenkins.

General

Webhooks / Manage webhook

Settings **Recent Deliveries**

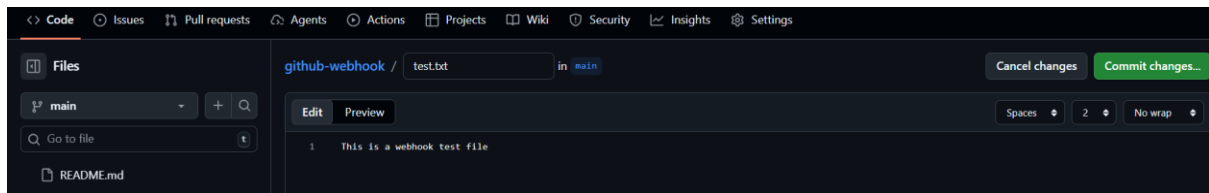
✓ c9e4121c-1d63-11f1-9631-b16413d9116c ping 2026-03-11 21:33:07 ...

Request Response **200** **Redeliver** ⌚ Completed in 0.52 seconds.

Headers

Request URL: http://3.216.30.120:8080/github-webhook/
 Request method: POST
 Accept: */*
 Content-Type: application/x-www-form-urlencoded
 User-Agent: GitHub-Hookshot/350c449
 X-Github-Delivery: c9e4121c-1d63-11f1-9631-b16413d9116c
 X-Github-Event: ping
 X-Github-Hook-Id: 600148617
 X-Github-Hook-Installation-Target-Id: 1178966228
 X-Github-Hook-Installation-Target-Type: repository

- Trigger Automatic Build
- Now make a change in GitHub.
- Example: Click Add File → Create new file.
- File name: test.txt
- Add content: This is a webhook test file.
- Click: Commit Changes



- Go to your Jenkins job.
- You will see a **new build triggered automatically**.

